

Double-click (or enter) to edit

Start coding or [generate](#) with AI.

mounting and importing

```
from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True)

```
import os

base_path = "/content/drive/MyDrive/project/dataset"
print(os.listdir(base_path))

['val', 'train', 'test']
```

copy data set to collab

```
!cp -r /content/drive/MyDrive/project/dataset /content/
```

```
!ls /content
```

```
config.yaml dataset drive sample_data
```

updating base path

```
base_path = "/content/dataset"
import os
print(os.listdir(base_path))

['test', 'train', 'val']
```

before everything: yaml running

```
%%writefile config.yaml
# Model
model_name: "DeepLabV3Plus"
encoder: "resnet50"
num_classes: 10

# Training
learning_rate: 0.0001
batch_size: 2
epochs: 15
image_size: 512

# Paths
train_images: "/content/dataset/train/Color_Images"
train_masks: "/content/dataset/train/Segmentation"
val_images: "/content/dataset/val/Color_Images"
val_masks: "/content/dataset/val/Segmentation"
```

Overwriting config.yaml

some more imports

```
import cv2
import matplotlib.pyplot as plt
import os
```

loading image

```
img_path = "/content/dataset/train/Color_Images/" + os.listdir("/content/dataset/train/Color_Images")[0]

img = cv2.imread(img_path)
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

plt.imshow(img)
```

```
plt.title("Sample Image")
plt.axis("off")
plt.show()
```

Sample Image



corresponding mask loading

```
mask_path = "/content/dataset/train/Segmentation/" + os.listdir("/content/dataset/train/Segmentation")[0]

mask = cv2.imread(mask_path, 0)

plt.imshow(mask, cmap='gray')
plt.title("Sample Mask")
plt.axis("off")
plt.show()
```

Sample Mask



class mapping

```
class_mapping = {
    100: 0,
    200: 1,
    300: 2,
    500: 3,
    550: 4,
    600: 5,
    700: 6,
    800: 7,
    7100: 8,
    10000: 9
}
```

```
label_mapping = {
    0: 0,
    1: 1,
    2: 2,
    3: 3,
    27: 4,
    39: 5
}
```

dataset class

```

import numpy as np
import torch
from torch.utils.data import Dataset

class DesertDataset(Dataset):
    def __init__(self, images_dir, masks_dir, transform=None):
        self.images = sorted(os.listdir(images_dir)) # 🔥 sorted (important)
        self.images_dir = images_dir
        self.masks_dir = masks_dir
        self.transform = transform

    def __len__(self):
        return len(self.images)

    def __getitem__(self, idx):
        img_name = self.images[idx]

        img_path = os.path.join(self.images_dir, img_name)
        mask_path = os.path.join(self.masks_dir, img_name)

        image = cv2.imread(img_path)
        image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

        mask = cv2.imread(mask_path, 0)

        # 🔥 map classes
        mask_mapped = np.zeros_like(mask)
        for k, v in label_mapping.items():
            mask_mapped[mask == k] = v

        # 🔥 augmentation
        if self.transform:
            augmented = self.transform(image=image, mask=mask_mapped)
            image = augmented["image"]
            mask_mapped = augmented["mask"]

        # 🔥 normalize
        image = image / 255.0
        image = np.transpose(image, (2, 0, 1))

        return torch.tensor(image, dtype=torch.float32), torch.tensor(mask_mapped, dtype=torch.long)

```

defining transforms

```

train_transform = A.Compose([
    A.Resize(512, 512),
    A.HorizontalFlip(p=0.5),
    A.RandomBrightnessContrast(p=0.5),
    A.HueSaturationValue(p=0.5),
    A.RandomGamma(p=0.5),
    # ❌ REMOVE THIS:
    # A.CoarseDropout(...)
])

val_transform = A.Compose([
    A.Resize(512, 512)
])

```

using 512*512 because its: Divisible by 16 ✓ Fast ✓ No DeepLab errors ✓

path definition

```

train_images = "/content/dataset/train/Color_Images"
train_masks = "/content/dataset/train/Segmentation"

val_images = "/content/dataset/val/Color_Images"
val_masks = "/content/dataset/val/Segmentation"

```

creating dataset objects

```

train_dataset = DesertDataset(train_images, train_masks, transform=train_transform)
val_dataset = DesertDataset(val_images, val_masks, transform=val_transform)

```

creating dataloaders

```

from torch.utils.data import DataLoader

train_loader = DataLoader(
    train_dataset,
    batch_size=2,
    shuffle=True,
    num_workers=2,
    drop_last=True # 🔥 prevents batchnorm error
)

val_loader = DataLoader(
    val_dataset,
    batch_size=2,
    shuffle=False,
    num_workers=2,
    drop_last=True
)

```

quikv sanity check

```
print(len(train_dataset), len(val_dataset))
```

```
2857 317
```

MODEL

```

import segmentation_models_pytorch as smp
import torch

```

model definition

```

model = smp.DeepLabV3Plus(
    encoder_name="resnet50",
    encoder_weights="imagenet",
    in_channels=3,
    classes=6
)

```

moving to gpu:

```

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)

```

```
print(device)
```

```
cuda
```

loss function

```
'''loss_fn = torch.nn.CrossEntropyLoss() -older one, changed to a newer one'''
```

```
'loss_fn = torch.nn.CrossEntropyLoss() -older one, changed to a newer one'
```

```

import torch.nn.functional as F

def dice_loss(pred, target, smooth=1e-6):
    pred = F.softmax(pred, dim=1)
    target_one_hot = F.one_hot(target, num_classes=6).permute(0,3,1,2).float()

    intersection = (pred * target_one_hot).sum(dim=(2,3))
    union = pred.sum(dim=(2,3)) + target_one_hot.sum(dim=(2,3))

    dice = (2. * intersection + smooth) / (union + smooth)
    return 1 - dice.mean()

def focal_loss(pred, target, alpha=0.25, gamma=2):
    ce_loss = F.cross_entropy(pred, target, reduction='none')
    pt = torch.exp(-ce_loss)
    focal = alpha * (1 - pt) ** gamma * ce_loss
    return focal.mean()

```

```
def hybrid_loss(pred, target):
    return focal_loss(pred, target) + dice_loss(pred, target)
```

```
loss_fn = hybrid_loss
```

optimizer

```
optimizer = torch.optim.Adam(model.parameters(), lr=1e-4)
```

Training

```
def train_model(train_loader):
    model.train()
    total_loss = 0

    for i, (images, masks) in enumerate(train_loader):

        if i % 20 == 0:
            print(f"Batch {i}")

        images = images.to(device)
        masks = masks.to(device)

        optimizer.zero_grad()
        outputs = model(images)

        loss = loss_fn(outputs, masks)
        loss.backward()
        optimizer.step()

        total_loss += loss.item() # ✅ correct accumulation

    return total_loss / len(train_loader)
```

pretraining

```
def calculate_iou(preds, masks, num_classes=6):
    ious = []

    preds = preds.view(-1)
    masks = masks.view(-1)

    for cls in range(num_classes):
        pred_inds = preds == cls
        target_inds = masks == cls

        intersection = (pred_inds & target_inds).sum().item()
        union = (pred_inds | target_inds).sum().item()

        if union == 0:
            continue

        ious.append(intersection / union)

    return sum(ious) / len(ious)
```

validation

```
def validate_model(val_loader):
    model.eval()
    total_loss = 0
    total_iou = 0
    with torch.no_grad():
        for images, masks in val_loader:
            images = images.to(device)
            masks = masks.to(device)

            outputs = model(images)
            loss = loss_fn(outputs, masks)
            preds = torch.argmax(outputs, dim=1)
            total_iou += calculate_iou(preds, masks)
            total_loss += loss.item() # ✅ correct

    return total_loss / len(val_loader), total_iou / len(val_loader)
```

run training

```

epochs = 10

for epoch in range(epochs):
    print(f"\nStarting Epoch {epoch+1}...")

    train_loss = train_model(train_loader)
    val_loss, val_iou = validate_model(val_loader)

    print(f"Epoch {epoch+1}")
    print(f"Train Loss: {train_loss:.4f}")
    print(f"Val Loss: {val_loss:.4f}")
    print(f"Val IoU: {val_iou:.4f}")    # 🔥 NEW LINE

```

Val IoU: 0.5058

Starting Epoch 2...

Batch 0
 Batch 20
 Batch 40
 Batch 60
 Batch 80
 Batch 100
 Batch 120
 Batch 140
 Batch 160
 Batch 180
 Batch 200
 Batch 220
 Batch 240
 Batch 260
 Batch 280
 Batch 300
 Batch 320
 Batch 340
 Batch 360
 Batch 380
 Batch 400
 Batch 420
 Batch 440
 Batch 460
 Batch 480
 Batch 500
 Batch 520
 Batch 540
 Batch 560
 Batch 580
 Batch 600
 Batch 620
 Batch 640
 Batch 660
 Batch 680
 Batch 700
 Batch 720
 Batch 740
 Batch 760
 Batch 780
 Batch 800
 Batch 820
 Batch 840
 Batch 860
 Batch 880
 Batch 900
 Batch 920
 Batch 940
 Batch 960
 Batch 980
 Batch 1000
 Batch 1020
 Batch 1040
 Batch 1060
 Batch 1080

visualisation

```

model.eval()

images, masks = next(iter(val_loader))

images = images.to(device)
outputs = model(images)

preds = torch.argmax(outputs, dim=1)

```

```

images = images.cpu().numpy()
masks = masks.cpu().numpy()
preds = preds.cpu().numpy()

import matplotlib.pyplot as plt

for i in range(len(images)):
    plt.figure(figsize=(15,5))

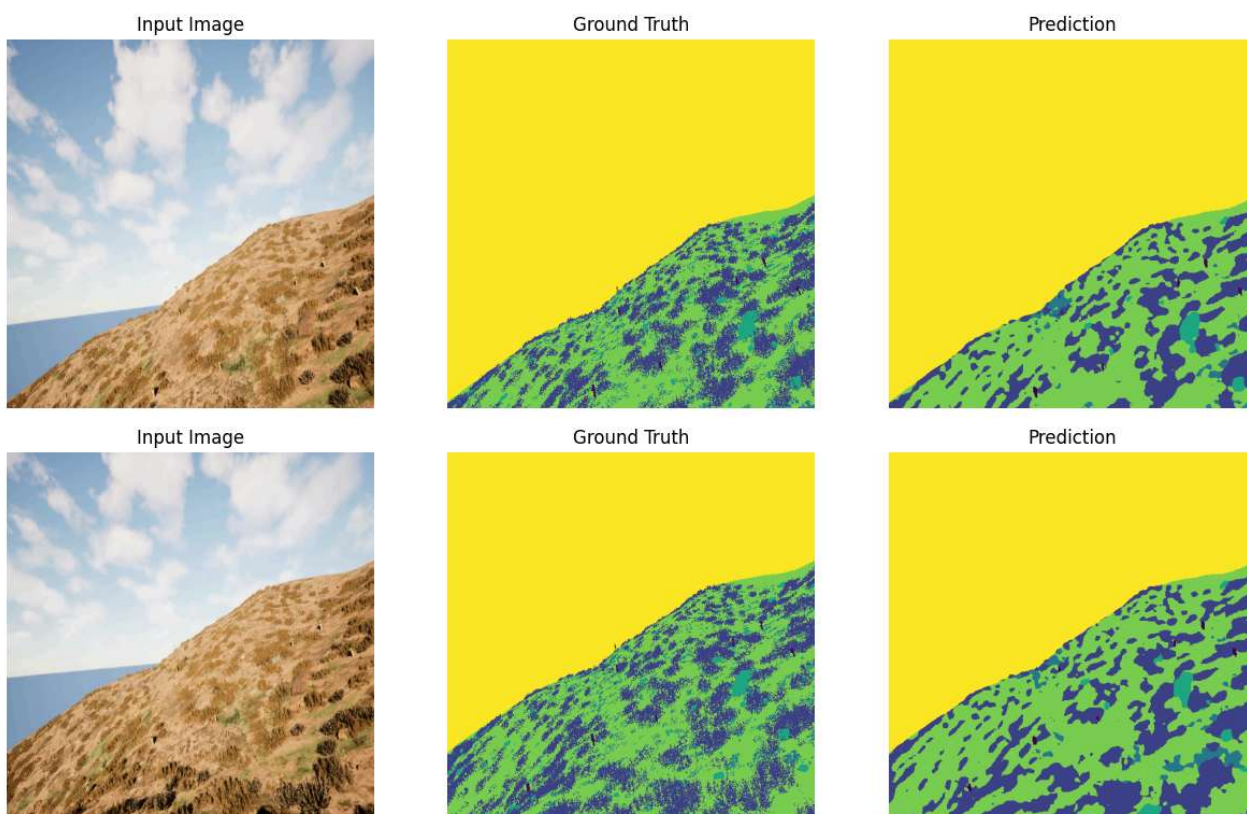
    # Image
    plt.subplot(1,3,1)
    plt.imshow(images[i].transpose(1,2,0))
    plt.title("Input Image")
    plt.axis("off")

    # Ground Truth
    plt.subplot(1,3,2)
    plt.imshow(masks[i])
    plt.title("Ground Truth")
    plt.axis("off")

    # Prediction
    plt.subplot(1,3,3)
    plt.imshow(preds[i])
    plt.title("Prediction")
    plt.axis("off")

plt.show()

```



saving images

```

import cv2
import os

os.makedirs("outputs", exist_ok=True)

for i in range(len(images)):
    pred_img = (preds[i] * 40).astype('uint8') # scale for visibility
    cv2.imwrite(f"outputs/pred_{i}.png", pred_img)

```

debugging

```
import numpy as np

sample_mask = cv2.imread("/content/dataset/train/Segmentation/" + os.listdir("/content/dataset/train/Segmentation")[0], 0)

print("Unique values in mask:", np.unique(sample_mask))
```

Unique values in mask: [0 1 2 3 27 39]

adding iou

```
def calculate_iou(preds, masks, num_classes=6):
    ious = []

    preds = preds.view(-1)
    masks = masks.view(-1)

    for cls in range(num_classes):
        pred_inds = preds == cls
        target_inds = masks == cls

        intersection = (pred_inds & target_inds).sum().item()
        union = (pred_inds | target_inds).sum().item()

        if union == 0:
            continue

        ious.append(intersection / union)

    return sum(ious) / len(ious)
```